

Decoder risk and finite-transfer tradeoffs in adaptive packet parity

A Aswanth Raj

Dr. Ranjithkumar S

School of Computer Science and Engineering
VIT Vellore
October 2026

Abstract

Adaptive packet parity depends on a loss model, a finite-sample estimate, and a transport objective. A validated decoder model alone cannot establish the reliability or goodput of the resulting transport. We study these layers in ParityLab, an open experimental artifact combining XOR/grid peeling, three protection policies, and four ARQ/FEC transport schemes. Across 153 configured independent or binary Markov conditions and 3,060,000 independently initialized decoder blocks, exact risk has maximum absolute prediction error 0.00914533, below the simultaneous 99% sampling tolerance of 0.01606919. Estimated targets remain fallible. On binary Markov data, eleven of thirty legacy-policy choices report feasibility at a 1% target while an individual measured 95% Wilson lower endpoint exceeds it. The burst policy declares twenty-nine of thirty cases infeasible, and its sole feasible choice is a failing independent fallback. At ten-percent independent loss in five-seed finite-file transfers, adaptive parity averages fewer retries than fixed XOR (7.6 ± 2.2 versus 9.6 ± 3.9) but lower goodput (1.154 ± 0.102 versus 1.247 ± 0.279 Mbps). A separate equal-budget paired-mask comparison changes the observed ordering of a small grid and replication between independent and burst erasures. Shared-queue and loopback studies verify additional capacity and protocol boundaries. The contribution is a reproducible empirical study that distinguishes decoder calibration, estimated feasibility, and useful throughput, with explicit negative results and model limits. It does not establish a new coding algorithm or an Internet performance guarantee.

1 Introduction

Forward erasure correction can recover a lost packet before a retransmission returns, but repairs consume capacity and may themselves be erased. Adaptive protection selects this cost using feedback, often reducing the channel to an estimated loss parameter. A reliability claim then has several premises: the probability calculation must match the decoder, the estimated parameters must describe the next transmission, and the chosen protection must serve the application’s timing or throughput objective. These premises are separate.

Existing research already addresses adaptive coding and application-specific recovery. AC-RLNC combines causal feedback with coding to examine throughput and in-order delay [4]. FIEC integrates application-tailored reliability into QUIC and evaluates bulk, buffer-limited, and deadline-constrained traffic [3]. Standards and research guidance also account for coding structure, redundancy costs, and congestion interaction [1, 7, 6]. Adaptive FEC is therefore established prior work. Our question concerns the evidence needed to evaluate a small decoder-aware controller and the failures that remain when its model is accurately implemented.

We examine an openly reproducible artifact spanning block decoding and finite-file transport.

Known-parameter decoder trials are separated from estimated-policy tests; matched repair-budget comparisons are separated from transport schedules; virtual-time simulation is separated from real loopback execution. This design makes a favorable coding probability insufficient by itself to support a transport claim. It also preserves simple baselines that remain competitive.

The measured findings are:

- Exact independent and binary Markov block models agree with byte-decoder sampling within the stated simultaneous tolerance, including repair erasures, conditional starting outcomes, and partial blocks (Section 5.2).
- Finite-history policies can report a risk target that the configured test process does not satisfy; a parameter envelope is conservative only when its model and parameter-inclusion premises hold (Section 5.3).
- Fewer mean retries can accompany lower mean useful goodput, and a matched-budget grid/replication comparison changes its observed ordering under burst erasures (Sections 5.1 and 5.4).

These are descriptive findings about the recorded conditions, supported by raw masks, transfer records, and source fingerprints. The artifact provides a controlled route for checking new code layouts or estimation rules. The paper does not claim an optimal adaptive policy, a historical priority result, or deployment-ready networking software.

2 Related work and positioning

2.1 Coding structure and residual loss

RFC 8681 specifies sliding-window random linear coding for FECFRAME [1]. Its coding family and framing context differ from the small XOR/grid block decoder evaluated here. Golaghazadeh, Coulombe, and Robert analyze residual packet loss for Pro-MPEG two-dimensional parity, derive bounds and approximations, and discuss burst-loss challenges [2]. This is the closest structural analysis: it already establishes that parity geometry and decoding deadlocks affect recovery. Our endpoint is the probability of *any* unrecovered source in a small block, evaluated by exact mask enumeration for the implemented peeling decoder. The empirical extension connects that endpoint to estimated protection choices and finite-transfer measurements. We do not present enumeration or row/column parity as a new coding construction.

2.2 Feedback and application objectives

AC-RLNC adapts random linear coding using delayed feedback and studies throughput and in-order delivery delay [4]. Its analysis and trace evaluation address stronger transport questions than a local four-source codec trial. FLEC exposes application-tailored reliability choices within QUIC and relates protection patterns to application needs [3]. It also evaluates burst behavior and loss estimates. The distinction here is an inspectable small-code calibration study: exact decoder risk, approximate estimate coverage, and finite-file goodput are audited as separate quantities. The current implementation is not a replacement for either coding system, and their published numbers are not treated as comparable baselines.

The expired 2024 adaptive-FEC Internet-Draft describes another delay-sensitive QUIC design direction [9]. Tsubaki and colleagues’ 2026 preprint uses per-path loss estimates, an expected-loss safety factor, Reed-Solomon coding, and commercial cellular field experiments [10]. Its controller explicitly favors implementation simplicity rather than optimizing block recovery probability under

binomial or burst loss. Those objectives and field conditions differ from this paper’s exact small-code risk and synthetic packet-index models. The preprint is concurrent context, with no peer-reviewed status assumed.

2.3 Redundancy, congestion, and evaluation boundaries

RFC 8854 describes FEC mechanisms and adaptive use in WebRTC, including the tradeoff between protecting data and increasing offered traffic [7]. RFC 9265 develops the relationship between FEC placement and congestion control, and recommends that coding comparisons consider congestion effects [6]. CloudBurst uses multipath FEC for datacenter tail latency with a commodity-switch testbed [8]. Its path diversity and workload differ from our single shared FIFO and finite files. Together, these sources establish that a coding result must be interpreted within an application and capacity regime; that principle is not a new discovery of this work.

The contribution is an empirical artifact with a documented chain from decoder equations to risk estimates to transport outcomes. It exposes counterexamples to treating a scalar loss estimate, a nominal target, or retry reduction as a sufficient performance claim. The contemporary bibliography is restricted to 2020-2026 as requested for this project. Searches covered adaptive coding, residual parity loss, model uncertainty, negative throughput effects, and adjacent application domains through OpenAlex, Crossref, arXiv, and primary full texts. Some automated graph queries were rate-limited; recent preprints and references from close works were checked separately. This bounded coverage supports positioning against the cited work without asserting exhaustive novelty.

3 Decoder and transport models

3.1 Symbol layout and residual block failure

ParityLab encodes equal-width byte symbols. A file is split into source symbols of width w , and the last symbol is zero-padded. The receiver removes the padding using the original file length. A protection configuration has an admitted source size k and a repair list E . A repair is the bitwise XOR of its member source symbols. For a partial tail of size $s \leq k$, empty equations are removed and all remaining equations refer only to the admitted source indices.

An XOR block carries one repair over its source symbols. A grid places sources in row-major order and sends one repair for each nonempty row, followed by one for each nonempty column. No repair over repair symbols is added. The decoder repeatedly resolves any surviving equation with exactly one unknown source. It stops when no such equation remains. Four erased sources at the corners of a rectangle are a stopping set when every relevant row and column contains two unknowns, even if every repair arrives. Peeling is the specified decoder throughout this study; a generic maximum-distance-separable recovery threshold is inappropriate for these codes.

The primary coding endpoint is residual block failure before retransmission: at least one original source remains unavailable after the first source-plus-repair transmission. This differs from residual *packet* loss rate, which averages missing source packets, and from eventual file delivery under ARQ. Let $\mathcal{F}$ be the erasure masks that cause peeling failure, let $n = s + |E|$, and let $h(e)$ count erased symbols in mask e . Repair erasures belong to the mask.

$$R_{\text{IID}}(p) = \sum_{e \in \mathcal{F}} p^{h(e)} (1-p)^{n-h(e)}. \quad (1)$$

For no repair, failure probability is $1 - (1-p)^s$. With one XOR repair it is $1 - (1-p)^s - sp(1-p)^s$: success requires all sources to arrive, or exactly one source to be erased while the repair arrives. Grid risk is evaluated by enumerating the actual peeling outcomes. The implementation limits

generic independent enumeration to sixteen transmitted symbols and binary Markov enumeration to twenty. The controller’s small candidate grids fit these bounds. Cached counts avoid repeating the structural enumeration for each probability value.

3.2 Dependence and the first transmitted outcome

The burst model is a binary first-order Markov erasure process. A good state receives a symbol and a bad state erases it. It is the deterministic-emission special case of a Gilbert-type channel, despite the configuration name `gilbert-elliott`. Let $a = P(B \mid G)$ and $b = P(G \mid B)$. When $a + b > 0$, stationary loss probability is $\pi = a/(a+b)$ and the mean bad-state run is $1/b$ transmitted symbols. Run length is indexed by forward packets, including repairs and retries, rather than elapsed time. Independent ACK erasures use a separate random stream.

Within a coding trial, wire order is all source symbols followed by the encoder’s repair list. For mask e , let e_1 be its first outcome and $n_{ij}(e)$ count transitions from i to j , with zero meaning receive and one meaning erase. Given first-symbol loss probability q ,

$$R_M(a, b, q) = \sum_{e \in \mathcal{F}} q^{e_1} (1 - q)^{1 - e_1} (1 - a)^{n_{00}(e)} a^{n_{01}(e)} b^{n_{10}(e)} (1 - b)^{n_{11}(e)}. \quad (2)$$

Stationary trials use $q = \pi$. Conditional trials use $q = a$ after a received preceding symbol, and $q = 1 - b$ after an erased preceding symbol. Every calibration block is independently initialized under its declared starting condition. Equal-budget burst trials instead keep the Markov state across successive blocks. These designs answer different questions. General hidden-state channels with probabilistic emissions, interleaved wire orders, and changing transition matrices are outside Equation (2)’s implemented model.

3.3 Protection policies and conditional uncertainty

The candidate set consists of no repair with capacity up to sixteen sources, XOR blocks of sizes sixteen, eight, four, and two, and grids of dimensions three-by-three, two-by-three, and two-by-two. Candidates exceeding the sender window are removed. A feasible choice has evaluated block risk at most $\tau = 0.01$. Among feasible candidates, the controller chooses the smallest repair-to-source ratio, breaking ties toward the larger admitted source block. If none is feasible, it chooses the lowest evaluated risk, breaks ties by repair ratio, and reports infeasibility. Infeasibility is not relabeled as successful protection.

The baseline adaptive scheme uses the legacy independent-loss policy. After a raw-loss report with l_t first-attempt losses among m_t reported transmissions, its estimate is

$$\hat{p}_t = (1 - \alpha)\hat{p}_{t-1} + \alpha l_t / m_t, \quad \alpha = 0.25, \quad \hat{p}_0 = 0. \quad (3)$$

Raw feedback reports pre-repair loss rather than the residual loss left after successful FEC. Otherwise, successful repair would make the channel appear cleaner than the observations justify. Legacy transport feedback counts first-attempt source outcomes. The optional ordered feedback includes first-attempt source and repair outcomes within a block and resets adjacency between reported blocks; it does not reconstruct missing intervals. The legacy transport policy evaluates its full candidate risk; the optional policies evaluate the actual admitted partial-block size. Zero-parity mode admits sources into the available sliding-window space without waiting for a full block, avoiding an artificial clean-link utilization penalty.

The uncertainty policy uses the upper endpoint of a two-sided approximate Wilson interval for recent raw loss counts. Complete feedback batches are retained with a default history budget of 512 observations; a single oversized batch is kept intact. Under an independent stationary model,

substituting an upper loss estimate into Equation (1) is conservative if that estimate actually exceeds the true loss probability. The approximate interval itself is not a deterministic guarantee of this condition.

The burst policy collects only adjacent ordered raw outcomes. An omitted wire interval resets adjacency; aggregate lost/sent counts cannot supply transitions. At least sixteen retained transitions and observations departing from both states are required. Otherwise the policy reports an independent uncertainty fallback. Eligible feedback produces estimated a, b and two approximate transition intervals, each with nominal confidence adjusted to 97.5% from the default 95% overall setting. The rectangular parameter set induces stationary initial-loss bounds. Its risk envelope sums the maximum probability of each failing-pattern class over the corresponding factor intervals.

For a nonnegative kernel $x^u(1-x)^v$, the maximum on an interval occurs at the interval-clipped value $u/(u+v)$ when $u+v > 0$; a constant kernel needs no optimization. Each exact mask probability is bounded by its factor maxima. Summing these nonnegative bounds and capping at one gives

$$R_M(a, b, q) \leq U(E, s; A, B, Q), \quad (a, b, q) \in A \times B \times Q. \quad (4)$$

Equation (4) is a conditional algebraic bound on the specified decoder and wire order. It makes no coverage claim for estimated rectangles. Maximizing factors separately can also yield a loose envelope that rejects every available protection choice. Calibration of known parameters and coverage of estimated parameters are therefore evaluated separately.

3.4 ARQ, serialization, and completion

Go-Back-N (GBN) accepts only the next source sequence at the receiver, emits cumulative acknowledgments, and retransmits the outstanding source window when its base timer expires. Selective Repeat (SR) buffers out-of-order sources, selectively acknowledges them, and uses per-source timers. Fixed XOR adds one repair per eight-source block to SR. Adaptive parity also uses SR for sources that remain unavailable after repair. Repairs are proactive; residual sources still have ARQ timers.

The emulator serializes forward and reverse directions independently, adds one-way propagation delay, and then applies configured erasure, jitter, and reordering rules. Modeled source/repair headers contribute forty bytes; ordinary ACKs contribute forty-eight bytes; modeled metadata contributes ninety-six bytes when used. These sizes are accounting assumptions. Receiver availability, contiguous application release, and sender acknowledgment completion have distinct timestamps. A lost early source can delay application release even when later sources are already buffered.

For source width w , window W , capacity C , one-way delay d , and maximum configured extra impairment $j+r$, the automatic source retransmission timer is

$$T_{\text{RTO}} = \max\{0.02, 2d + 2(j+r) + (W+16)8(w+40)/(10^6C) + 0.005\} \text{ s}. \quad (5)$$

Here C is in Mbps and delays on the right are in seconds. The baseline uses this deterministic timer, without RTT adaptation or fast loss detection. Explicit timers are examined in the sensitivity study. The default source event budget is 500,000. A bounded run that exhausts its budget is incomplete rather than a successful file transfer. Baseline block descriptors use an idealized out-of-band path. Raw feedback is serialized on the reverse path and can be erased; its payload size and content are modeled rather than taken from a wire encoding. Reliable impaired metadata is tested separately. The real socket protocol impairs all message types.

3.5 Useful throughput and competing flows

For a complete file of original length L , useful goodput is $8L/(10^6 t_R)$ Mbps, with t_R the receiver-availability completion time. Sender completion includes acknowledgment wait. Recovery delay is source availability minus that source’s first transmission start; application delay uses contiguous release instead. Packet p95 values use the nearest-rank order statistic within a transfer. A mean of per-transfer p95 values is not a pooled packet p95.

Parity overhead is transmitted repair payload divided by original file bytes; repair padding is included. Forward wire overhead also includes source padding, modeled headers, and retransmitted sources. Retry count records each source send after its first attempt, including unnecessary sends caused by ACK loss. FEC recovery ratio is the fraction of first-attempt source losses recovered before a retransmission is sent; it is defined as zero when no such losses occur.

The competing-flow emulator adds one finite shared FIFO with tail drop. Queue capacity includes the packet in service. Source, repair, and retransmission packets consume the same per-flow flight budget. AIMD flows start with four flight slots, have a maximum configured window of sixty-four, and begin with a slow-start threshold of sixty-four. An ACK increases the window by one below the threshold and by its reciprocal above it, capped at sixty-four. A timeout halves the window with a minimum of one and resets the threshold to at least two. These are teaching rules, not a deployed congestion controller. Congestion flight slots remain separate from GBN reliability sequence timers. Actual useful in-order release rates x_1, x_2 over the common active interval give

$$J = (x_1 + x_2)^2 / [2(x_1^2 + x_2^2)]. \quad (6)$$

The interval excludes periods when one flow has finished. Finite-file starts and tails still affect measured sharing. Aggregate goodput and this common-interval index are separate endpoints. The teaching controller does not implement RFC 9002’s complete QUIC loss detection or congestion behavior [5].

4 Experimental design

4.1 Status, replication, and sample units

This is a retrospective descriptive analysis of a completed experimental artifact. The controller candidates, defaults, and recorded results existed before this manuscript framing. No study was preregistered, no reporting split was held out, and no prospective power calculation determined the budgets. Configurations were selected for a networks-laboratory comparison and implementation checks, rather than by a systematic tuning competition. The optional policies were added as explicit alternatives; they were not silently substituted into the baseline adaptive results.

Table 1 lists the separate sample units. Counts must not be pooled into one effective sample size. Transport summaries report arithmetic means and sample standard deviations across seeds, using divisor $n - 1$. The seed labels repeat across methods but the transport packet schedules do not: they consume different random draws. These are common *configured seeds*, not paired erasure traces. We use no transport paired significance test and claim no uniform ranking. Only the equal-budget codec comparisons explicitly share masks within a pair.

Table 1. Separate studies and observation units. Each shared mask is decoded by two equal-budget codes, giving 160,000 codec evaluations. Policy validation contains 225 policy decisions from seventy-five training histories shared across three policies, followed by 225,000 test blocks. Neither the shared histories nor repeated test blocks supply independent controller estimates for every row.

Study	Observation unit	Count	Replication
Single-flow	File transfer	260	5 seeds/condition/scheme
Sensitivity	File transfer	370	5 seeds/condition/scheme
Equal budget	Shared mask	80000	5 seeds; 1000 masks/group/seed
Decoder calibration	Independent block draw	3060000	153 cases; 5 x 4000 blocks/case
Estimated policy	Policy decision	225	75 shared histories; 3 policies
Policy decoding	Fresh test block	225000	1000 draws/training realization
Shared FIFO	Two-flow run	70	5 seeds/configuration/queue
Loopback	Three-process transfer	36	3 seeds/scenario/scheme

4.2 Single-flow and sensitivity settings

The baseline transfers 128 KiB of deterministic binary payload, split into 1 KiB source symbols, with window 32, capacity 5 Mbps, and 50 ms one-way delay. Five seeds, zero through four, are evaluated for each of four schemes. Eleven independent loss settings run from zero to twenty percent in two-percentage-point steps. Reverse ACK loss, jitter, and reordering are disabled in these baseline cases. The burst condition sets stationary loss to ten percent and $b = 0.2$. Its expected bad-state run is five forward symbols. The changing-loss condition transfers 512 KiB: it starts clean, then uses loss probabilities 0.15, 0.02, and 0.10 from virtual times 0.4, 1.2, and 2.0 seconds. Loss phases apply by transmission serialization end time. All configured baseline runs are retained.

Sensitivity uses 64 KiB files and changes one factor from independent ten-percent loss, 50 ms delay, 5 Mbps capacity, window 32, 1 KiB sources, and fixed XOR size eight. Tested delays are zero, twenty-five, and one hundred milliseconds; capacities are one, two, and ten Mbps; windows are eight, sixteen, and sixty-four; explicit timeouts are seventy, one hundred eighty, and five hundred milliseconds; symbol widths are 512 and 1,200 bytes. Fixed-block sizes four and sixteen are tested only for fixed XOR. Binary Markov mean bad-state runs two, five, and ten use the same ten-percent stationary loss. Unchanged baseline settings are also retained. Five seeds per applicable scheme produce 370 transfers. Appendix B reports every condition.

4.3 Paired decoder comparisons at fixed redundancy

Four source symbols are sent before all repairs, with no retransmission. At a 100% payload budget, a two-by-two row/column grid and four per-source replicas each send four repairs. At a 50% budget, two disjoint pairwise XOR equations and two copies of a whole-block XOR each send two repairs. Equal-width payloads, symbol count, order, and erasure mask are shared within each pair. The test uses independent and binary Markov erasures at loss probabilities 0.01, 0.05, 0.10, and 0.20, five seeds, and one thousand consecutive blocks per seed/group. Markov $b = 0.2$; initial channel state is stationary and then persists across blocks.

Paired seed-level contrasts average the within-seed difference in failure rates; their uncertainty is the sample SD of the five contrasts. This preserves the pairing while treating whole seeded traces as the replication units. We do not use per-block binomial coverage for correlated burst traces. Stored Wilson columns for that study are descriptive only. The code comparison concerns the chosen ordering and four-source layouts; it is not a claim about all grid sizes or a transport throughput benchmark.

4.4 Known-parameter calibration

Nine independent-code configurations span no repair, XOR sizes sixteen, eight and two, grids three-by-three, two-by-three and two-by-two, and partial-grid source sizes five and one. Loss probabilities are zero, 0.01, 0.03, 0.10, 0.20, 0.50, and one, producing sixty-three independent conditions. Five burst-code configurations combine three stationary loss probabilities (0.03, 0.10, 0.20), two b values (0.2, 0.5), and three starting conditions (stationary, after receive, after loss), producing ninety binary Markov conditions. Each of the 153 conditions uses five seeds and four thousand independently initialized block draws per seed. Source and repair erasures are recorded along with byte-decoder outcomes. The small diagnostic source payloads contain distinct values, a zero byte, and a 255 byte; broader binary-file checks belong to the transport suite.

For each condition we report the exact model probability, empirical failure fraction, and an approximate two-sided 95% Wilson sampling interval. To distinguish individual interval misses from a simultaneous check, Hoeffding’s inequality and a union bound over $M = 153$ conditions, each with $N = 20,000$ draws, give

$$\epsilon = \sqrt{\log(2M/0.01)/(2N)} = 0.01606919. \quad (7)$$

Under the independent-block sampling design, the probability that any empirical fraction differs from its true probability by more than ϵ is at most 0.01. This check audits known-parameter prediction against the implemented byte decoder. It does not validate inferred transition intervals or the online transport controller. Seeded pseudorandom sampling approximates the modeled independent draws.

4.5 Estimated policies and model mismatch

Each controller is trained on a raw history of sixty-four, 512, or 4,096 outcomes, with five seeds for each history and model. Independent training/testing uses loss 0.03 and 0.10; binary Markov uses $(\pi, b) = (0.10, 0.2)$ and $(0.20, 0.5)$. A deterministic-duration challenge repeats ninety receives followed by ten erasures, with a randomly selected starting offset. The three policies share the same training sequence within a realization. For this isolated estimation experiment, the legacy smoothing factor is one and its estimate is the entire supplied history’s loss fraction. This setting differs from the baseline transport’s online $\alpha = 0.25$ updates.

One thousand fresh blocks per policy decision are drawn from the declared stationary model, or from independent starting offsets in the deterministic-duration challenge. The chosen protection can differ across policies, so shared test-generator seeds do not imply identical symbol masks. A diagnostic target failure is counted when the policy reports feasibility but the empirical failure rate’s individual 95% Wilson lower endpoint exceeds 0.01. This threshold is a descriptive diagnostic across many correlated training comparisons; no family-wise decision rule is asserted for those counts. A separate diagnostic compares evaluated risk directly with the known exact risk when the model is independent or binary Markov.

4.6 Shared queue and socket conditions

Shared-queue runs transfer two 256 KiB files through 2 Mbps capacity and 20 ms one-way delay, with independent channel loss and ACK loss each 0.01. Queue capacities are eight and thirty-two packets, with five seeds per configuration. Equal AIMD SR flows, equal uncontrolled SR flows, and an AIMD-versus-uncontrolled pair are evaluated. Each of the four reliability schemes also competes with AIMD SR. The `aimd_equal` and `sr_vs_sr` configurations are intentional repeats of the same specification and must not be counted as independent new evidence for that comparison.

The seventy recorded runs contain 140 completed flows. Descriptors remain out of band in this evaluator.

The socket study sends 32 KiB files in 512-byte sources using separate sender, impairment-proxy, and receiver processes bound to loopback. All manifests, block descriptors, raw-loss feedback, source/repair data, ACKs, and final confirmations traverse the proxy. Independent loss uses forward loss 0.15 and ACK loss 0.10; the burst case uses forward stationary loss 0.10, $b = 0.2$, and ACK loss 0.05. A combined impairment case uses source loss 0.05, ACK loss 0.10, metadata loss 0.40, uniform jitter up to three milliseconds, reordering probability 0.20 with ten milliseconds extra delay, duplication probability 0.10, and corruption probability 0.02. All use two milliseconds base one-way delay. Three seeds and four schemes give thirty-six transfers. The per-run wall-clock budget is twenty seconds and the configured per-message attempt bound is 150.

CRC-framed datagrams reject accidental corruption. Retried metadata and a final SHA-256 exchange verify complete output bytes. Framed UDP payload bytes are measured, excluding IP and UDP headers. Process scheduling changes real arrival order and timing. Socket wall time is kept separate from emulator virtual time; the socket results support protocol execution and byte integrity within the tested conditions.

5 Results

5.1 Parity reduces some retries while consuming useful capacity

Table 2 reports selected baseline conditions and Figure 1 includes every independent-loss setting. At ten-percent independent loss, fixed XOR goodput is 1.247 ± 0.279 Mbps and adaptive goodput is 1.154 ± 0.102 Mbps across five seeds. Their retries average 9.6 ± 3.9 and 7.6 ± 2.2 , respectively. Adaptive therefore has fewer mean retries in these saved runs while its mean goodput is lower. Adaptive parity/file payload is 0.344 ± 0.178 , compared with fixed XOR's 0.125 ± 0.000 . These summaries are descriptive; the experiment does not establish a statistically resolved population ranking.

At twenty-percent independent loss, adaptive goodput is 1.038 ± 0.323 Mbps versus 0.731 ± 0.076 Mbps for fixed XOR, with parity/file ratios of 0.606 ± 0.124 and 0.125 ± 0.000 . Clean-link fixed XOR pays its proactive repair cost despite zero source erasures. In the burst and changing-loss cases, adaptive mean goodput remains below fixed XOR's mean. Both mixed and favorable comparisons are retained.

Table 2. Receiver-availability goodput, data retries and parity overhead. Entries are mean $\pm$ sample SD over five seeds. IID is independent identically distributed erasure loss; Markov is the binary five-symbol mean-burst condition. The changing-loss file is four times larger than the other files, so cross-condition goodput differences are not a controlled file-size comparison.

Condition	Scheme	Goodput (Mbps)	Data retries	Parity/file
IID 0%	GBN	2.559 ± 0.000	0.0 ± 0.0	0.000 ± 0.000
IID 0%	SR	2.559 ± 0.000	0.0 ± 0.0	0.000 ± 0.000
IID 0%	Fixed XOR	2.327 ± 0.000	0.0 ± 0.0	0.125 ± 0.000
IID 0%	Adaptive	2.559 ± 0.000	0.0 ± 0.0	0.000 ± 0.000
IID 10%	GBN	0.388 ± 0.053	356.0 ± 80.9	0.000 ± 0.000
IID 10%	SR	0.874 ± 0.202	15.6 ± 3.6	0.000 ± 0.000
IID 10%	Fixed XOR	1.247 ± 0.279	9.6 ± 3.9	0.125 ± 0.000
IID 10%	Adaptive	1.154 ± 0.102	7.6 ± 2.2	0.344 ± 0.178
IID 20%	GBN	0.185 ± 0.039	831.8 ± 163.6	0.000 ± 0.000
IID 20%	SR	0.675 ± 0.043	31.4 ± 4.6	0.000 ± 0.000
IID 20%	Fixed XOR	0.731 ± 0.076	23.6 ± 5.9	0.125 ± 0.000
IID 20%	Adaptive	1.038 ± 0.323	11.0 ± 3.1	0.606 ± 0.124
Markov 10%	GBN	1.124 ± 0.446	83.8 ± 67.3	0.000 ± 0.000
Markov 10%	SR	1.479 ± 0.317	10.6 ± 6.4	0.000 ± 0.000
Markov 10%	Fixed XOR	1.465 ± 0.244	8.4 ± 4.9	0.125 ± 0.000
Markov 10%	Adaptive	1.443 ± 0.312	9.6 ± 4.9	0.106 ± 0.161
Changing	GBN	0.614 ± 0.088	907.6 ± 195.5	0.000 ± 0.000
Changing	SR	1.210 ± 0.171	32.4 ± 11.1	0.000 ± 0.000
Changing	Fixed XOR	1.602 ± 0.141	16.6 ± 6.1	0.125 ± 0.000
Changing	Adaptive	1.556 ± 0.087	13.6 ± 3.5	0.318 ± 0.045

5.2 Exact block models agree with the byte decoder

Across 3,060,000 independently initialized decoder blocks in 153 conditions, the byte checks found zero incorrectly recovered source values. Maximum absolute prediction error is 0.00914533. Every exact prediction lies within the simultaneous 99% sampling tolerance of 0.01606919 from Equation (7). Five predictions fall outside their individual approximate 95% Wilson intervals. The individual misses and simultaneous check concern different coverage statements and are reported together.

Figure 2 shows all configured exact/empirical probability pairs. Table 3 isolates the effect of the starting condition at ten-percent stationary loss. For XOR over eight sources with $b = 0.2$, exact stationary block risk is 0.1894, risk after receive is 0.1342, and risk after loss is 0.6870. The corresponding independent approximation uses 0.2252 for all starting conditions. Maximum error against the measured failure fractions of that independent approximation across stationary burst conditions is 0.18469238; including conditional starts raises the maximum to 0.63695400. These are absolute probability errors within the configured calibration cases, without an extrapolation interval.

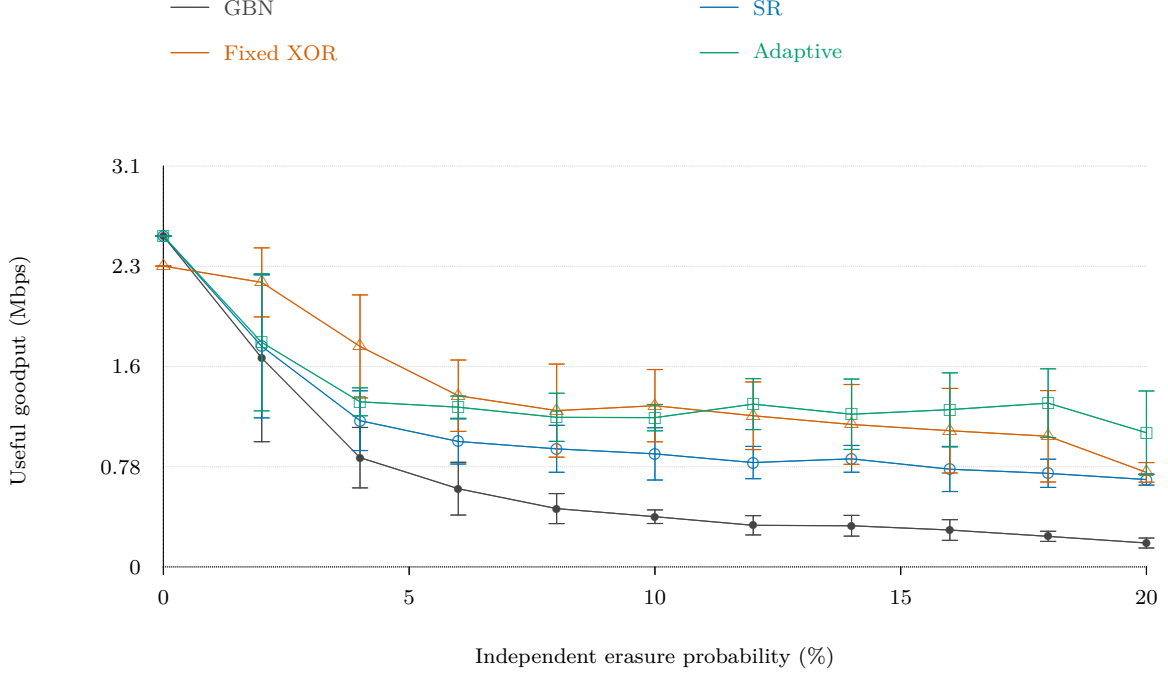


Figure 1: Mean goodput under every independent-loss setting. Error bars are sample SD over five seeds per scheme/condition, with 128 KiB files, 1 KiB symbols, window 32, 5 Mbps capacity and 50 ms one-way delay. Adaptive uses the legacy independent-loss policy. The curves show condition-specific tradeoffs; seed schedules are not paired erasure traces.

Table 3. Selected known-parameter calibration cases. Each row has 20,000 independently initialized blocks; intervals are approximate individual 95% Wilson sampling intervals. Independent rows use loss 0.10; binary Markov rows use stationary loss 0.10 and $b = 0.2$. Grid rows with five data symbols are partial tails of a three-by-three candidate. Failure means at least one original source is missing before ARQ.

Start/model	Code	Data	Exact risk	Failure rate	95% Wilson interval
IID	xor-8	8	0.2252	0.2251	[0.2194, 0.2310]
IID	grid-3x3	9	0.0114	0.0113	[0.0099, 0.0129]
IID	grid-3x3	5	0.0056	0.0057	[0.0048, 0.0069]
Stationary	xor-8	8	0.1894	0.1913	[0.1859, 0.1968]
Stationary	grid-3x3	9	0.1025	0.1013	[0.0972, 0.1056]
Stationary	grid-3x3	5	0.0748	0.0747	[0.0712, 0.0785]
After receive	xor-8	8	0.1342	0.1339	[0.1293, 0.1387]
After receive	grid-3x3	9	0.0701	0.0698	[0.0664, 0.0735]
After receive	grid-3x3	5	0.0426	0.0425	[0.0397, 0.0453]
After loss	xor-8	8	0.6870	0.6907	[0.6843, 0.6971]
After loss	grid-3x3	9	0.3939	0.3932	[0.3865, 0.4000]
After loss	grid-3x3	5	0.3648	0.3658	[0.3591, 0.3725]

5.3 Estimated feasibility is weaker than known-model calibration

Table 4 reports every model/policy group. For binary Markov data, the legacy estimate falls below true modeled risk in 28 of thirty cases, and eleven cases report feasibility while the individual measured lower endpoint exceeds the target. The uncertainty policy has twenty risk underestimates

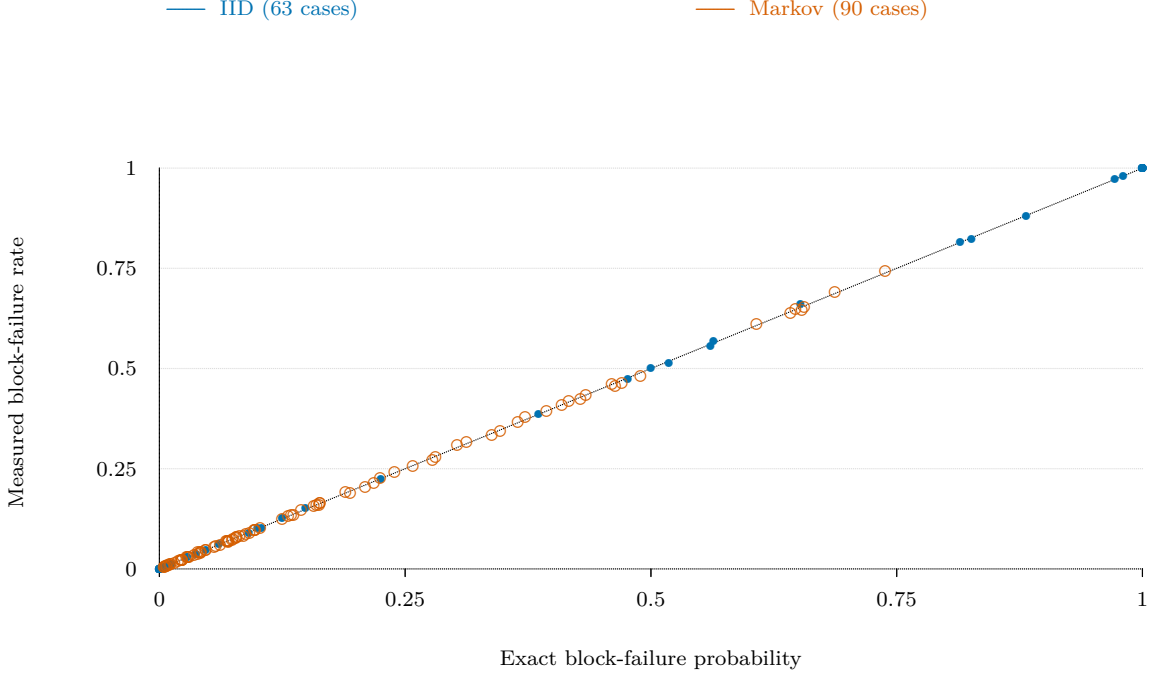


Figure 2: Exact predictions and observed block-failure fractions for all 153 calibration conditions. Each point aggregates 20,000 independently initialized blocks; the diagonal is equality. The 63 independent and 90 binary Markov conditions include endpoints and partial blocks. Maximum absolute deviation is 0.00914533; the simultaneous 99% Hoeffding tolerance is 0.01606919.

and seven diagnostic target failures among thirty cases. The burst policy has one of each among thirty cases, but it declares twenty-nine cases infeasible. Its single feasible choice uses the independent fallback and fails the target diagnostic. The lower failure count therefore reflects abstention rather than twenty-nine demonstrated successful target decisions. These counts describe the saved policy cases and do not estimate an unconditional deployment failure probability.

Twenty-nine binary Markov burst-policy cases use an active parameter envelope; one falls back to the independent uncertainty model. One of the twenty-nine active rectangles excludes the true transition parameters. Among the twenty-eight that contain them, zero evaluated envelopes fall below exact stationary risk. The excluded active rectangle also returns a conservative risk of one and declares infeasibility; the observed underestimate comes from the fallback case. The conditional algebraic bound and the finite coverage check agree, while the missed rectangle remains a coverage failure. The deterministic-duration challenge yields thirteen, twelve, and one target failures among fifteen cases for legacy, uncertainty, and burst policies, respectively. A binary Markov fit does not make that challenge a binary Markov channel.

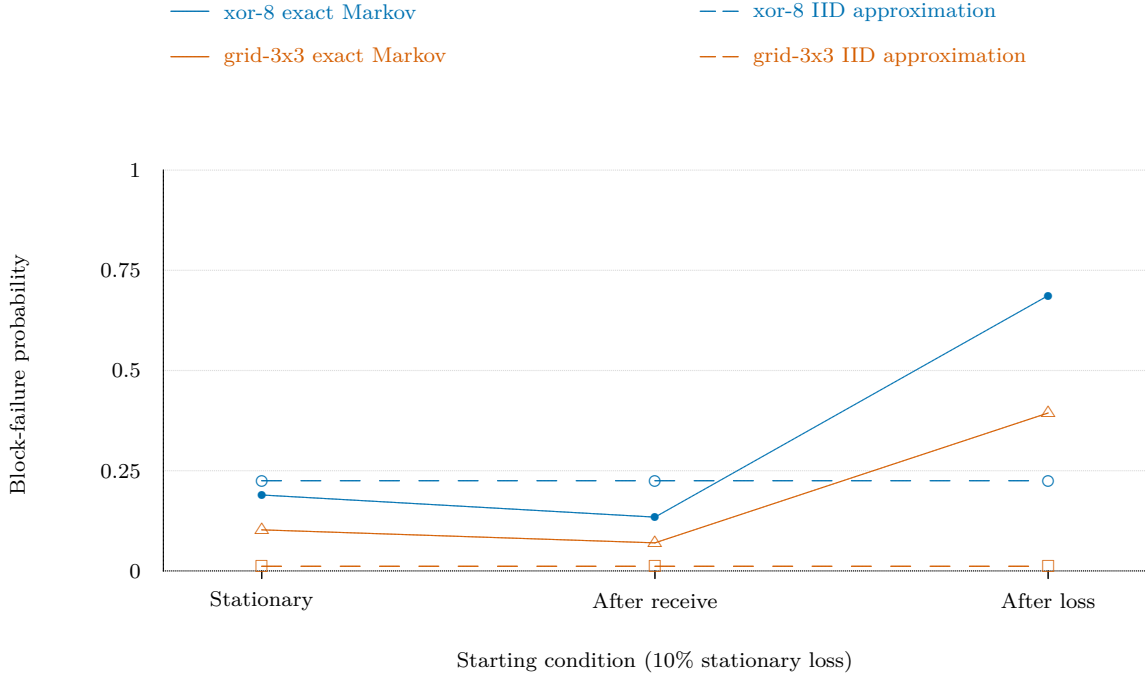


Figure 3: A preceding loss changes the next block’s modeled risk. Exact binary Markov risk uses stationary loss 0.10, bad-to-good probability 0.2, sources before repairs, and the specified peeling decoder. The IID approximation ignores the preceding outcome and stays constant. These are model probabilities, with no empirical uncertainty; Table 3 reports the independently sampled decoder checks.

Table 4. Estimated-policy diagnostics. Cases are policy decisions pooled within the named model/policy group; the same histories are reused across policies, with five seeds at each history/parameter setting. Feasible choices report evaluated risk at most 1%. Risk underestimates compare to known exact model risk; N/A means that reference is not defined for the fixed-run challenge. Target failures count feasible choices whose individual 95% Wilson lower endpoint exceeds 1%. These are descriptive multiple-comparison counts, not simultaneous coverage guarantees.

Training/test model	Policy	Cases	Feasible choices	Risk underestimates	Target failures
independent	legacy	30	27	13	2
independent	uncertainty	30	20	0	0
independent	burst	30	14	0	0
binary_markov	legacy	30	11	28	11
binary_markov	uncertainty	30	7	20	7
binary_markov	burst	30	1	1	1
fixed_runs	legacy	15	13	N/A	13
fixed_runs	uncertainty	15	12	N/A	12
fixed_runs	burst	15	1	N/A	1

5.4 Equal-budget decoder ordering reverses under bursts

At ten-percent independent loss and a 100% repair budget, the grid fails in 28 of 5,000 shared-mask blocks, while per-source replication fails in 203 of 5,000. At the same configured stationary loss under continuous binary Markov erasures, their failures are 380 and 340 of 5,000. Table 5 reports

counts for every loss setting at that budget. Figure 4 plots paired seed contrasts with sample SD. The observed grid advantage under independent loss reverses under this burst ordering. The four-source fixed layout and consecutive source-then-repair schedule are part of that finding.

At the 50% budget, the two pairwise XOR equations and repeated whole-block XOR are close in several conditions. At ten-percent independent loss they fail in 273 and 298 of 5,000 blocks; under bursts, they fail in 573 and 595 of 5,000. Appendix C preserves all of those counts. Neither budget comparison optimizes a transport controller or measures retransmission timing.

Table 5. Paired masks at the 100% repair budget. Each code decodes the same 5,000 masks per model/loss group, divided into five seeded traces. The last column is the mean $\pm$ sample SD of paired seed-level failure-rate differences in percentage points (pp). A positive value means more grid block failures. Burst blocks within a seed are correlated.

Model	Loss	Grid failures	Replication failures	Grid - replication (pp)
IID	1%	0/5000	1/5000	-0.02 $\pm$ 0.04
IID	5%	2/5000	50/5000	-0.96 $\pm$ 0.38
IID	10%	28/5000	203/5000	-3.50 $\pm$ 0.80
IID	20%	193/5000	747/5000	-11.08 $\pm$ 1.35
Markov	1%	31/5000	27/5000	0.08 $\pm$ 0.08
Markov	5%	201/5000	180/5000	0.42 $\pm$ 0.24
Markov	10%	380/5000	340/5000	0.80 $\pm$ 0.29
Markov	20%	810/5000	751/5000	1.18 $\pm$ 0.19

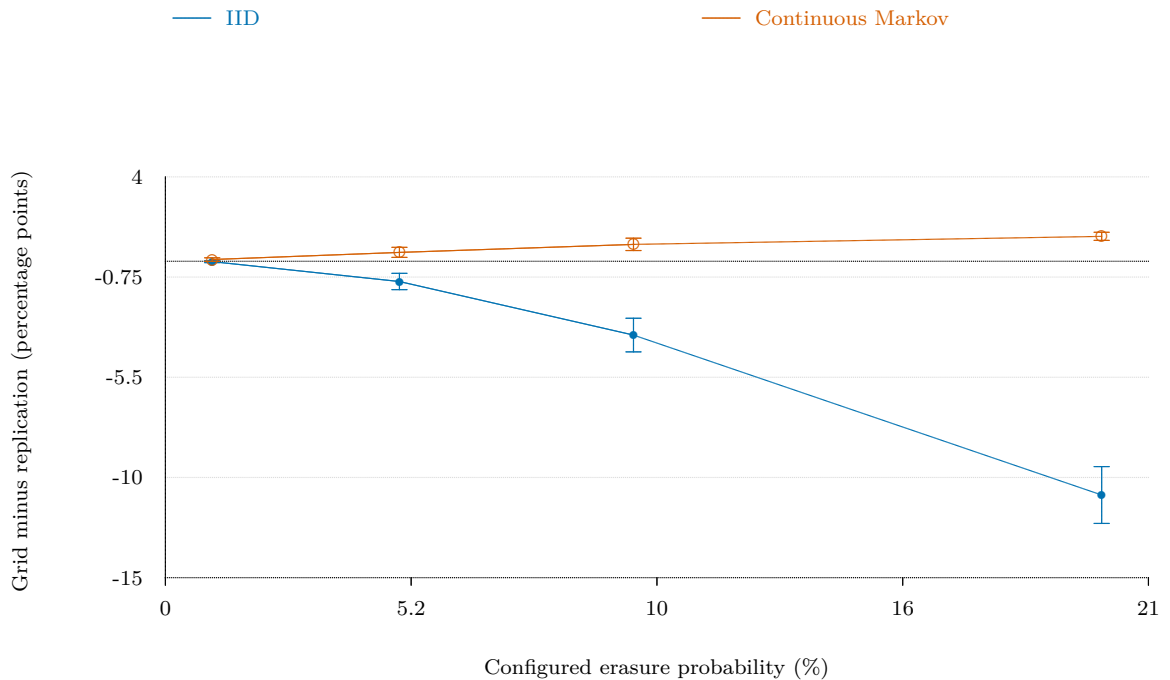


Figure 4: Paired failure-rate contrasts for the two-by-two grid and per-source replication with four source plus four repair symbols. Negative values favor the grid. Error bars show sample SD of five seed-level contrasts, each based on 1,000 shared masks. Bursts preserve Markov state across blocks and use a five-symbol mean bad run. The axes include all configured contrasts.

5.5 Shared capacity and reliable control expose separate costs

All seventy shared-queue runs complete both files with verified output bytes. At queue capacity eight, equal AIMD SR goodput is 1.765 ± 0.055 Mbps and equal uncontrolled SR goodput is 1.381 ± 0.049 Mbps. Their mean queue-drop counts are 34.2 and 1,768.0. Figure 5 compares useful capacity under the recorded repair/flight accounting. Table 6 includes all configurations and their common-interval Jain indices. Fixed XOR versus SR and adaptive versus SR have lower mean aggregate goodput than SR versus SR at both saved queue capacities. These finite-file measurements belong to the teaching controller, without a TCP/QUIC compatibility claim.

All thirty-six recorded socket transfers verify their 32 KiB output and use three distinct processes. Table 7 reports retries and repair packets across three seeds per scenario/scheme. The combined impairment condition completes with metadata erasure, reordering, duplication, and corruption enabled. A separate correctness test for permanent metadata loss checks bounded failure and absence of a successful output file; that failure case is not silently included among the thirty-six completed study runs. CRC and digest validation detect accidental transmission or reconstruction errors in these tests, without authenticating a peer.

Table 6. Shared FIFO results. Each row reports five seeded two-flow runs. Goodput and Jain index are mean $\pm$ sample SD; queue drops are the mean count per run. Queue size includes the in-service packet. `aimd equal` and `sr vs sr` duplicate the same configuration and are not independent evidence. Jain index uses common-interval useful in-order release, whereas aggregate goodput divides both files’ useful bits by the span from the earliest flow start to the latest receiver completion.

Two-flow configuration	Queue	Goodput (Mbps)	Jain index	Queue drops
aimd equal	8	1.765 ± 0.055	0.961 ± 0.055	34.2
aimd equal	32	1.734 ± 0.065	0.957 ± 0.031	48.0
uncontrolled equal	8	1.381 ± 0.049	0.815 ± 0.107	1768.0
uncontrolled equal	32	1.697 ± 0.089	0.928 ± 0.116	512.6
aimd vs uncontrolled	8	1.549 ± 0.177	0.959 ± 0.049	742.8
aimd vs uncontrolled	32	1.586 ± 0.150	0.772 ± 0.176	141.6
gbn vs sr	8	1.172 ± 0.119	0.787 ± 0.078	32.2
gbn vs sr	32	1.348 ± 0.181	0.905 ± 0.081	48.2
sr vs sr	8	1.765 ± 0.055	0.961 ± 0.055	34.2
sr vs sr	32	1.734 ± 0.065	0.957 ± 0.031	48.0
fixed vs sr	8	1.662 ± 0.038	0.956 ± 0.062	34.4
fixed vs sr	32	1.710 ± 0.033	0.960 ± 0.013	48.2
adaptive vs sr	8	1.552 ± 0.040	0.952 ± 0.056	36.6
adaptive vs sr	32	1.662 ± 0.105	0.993 ± 0.005	47.0

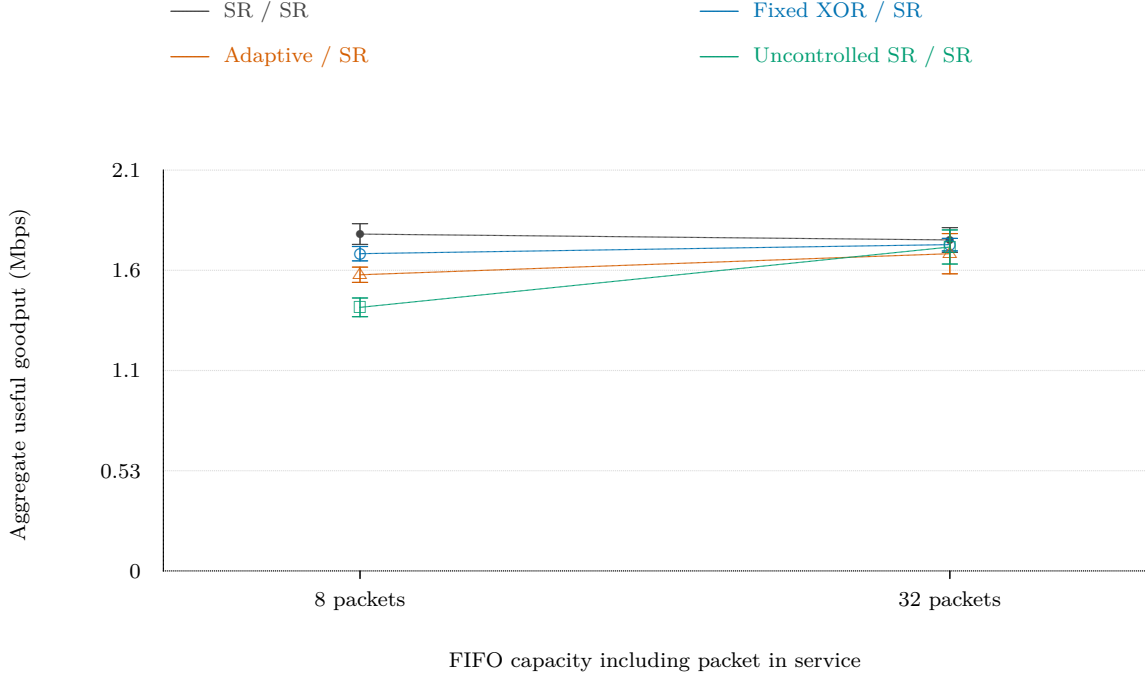


Figure 5: Aggregate useful goodput for selected two-flow configurations at each queue capacity. Error bars are sample SD over five seeds. The bottleneck is 2 Mbps with 20 ms one-way delay and 1% channel/ACK loss; each flow sends 256 KiB. Data, repairs and retries share the flight budget. These results are from a teaching AIMD emulator.

Table 7. Recorded loopback protocol execution. Retry and repair counts are mean $\pm$ sample SD over three seeds. Byte-verified counts require exact output and digest agreement. Every scenario uses impaired control messages and three distinct processes; timing and loss masks are subject to operating-system scheduling. Wall time is deliberately excluded from this table’s comparison.

Scenario	Scheme	Byte-verified	Retries	Repair packets
independent	GBN	3/3	156.0 $\pm$ 86.0	0.0 $\pm$ 0.0
independent	SR	3/3	17.0 $\pm$ 3.0	0.0 $\pm$ 0.0
independent	Fixed XOR	3/3	14.7 $\pm$ 3.8	8.0 $\pm$ 0.0
independent	Adaptive	3/3	15.7 $\pm$ 0.6	18.3 $\pm$ 11.6
burst	GBN	3/3	25.7 $\pm$ 23.5	0.0 $\pm$ 0.0
burst	SR	3/3	10.3 $\pm$ 5.0	0.0 $\pm$ 0.0
burst	Fixed XOR	3/3	9.7 $\pm$ 4.5	8.0 $\pm$ 0.0
burst	Adaptive	3/3	9.7 $\pm$ 4.7	10.3 $\pm$ 17.9
metadata reorder	GBN	3/3	674.0 $\pm$ 240.0	0.0 $\pm$ 0.0
metadata reorder	SR	3/3	15.3 $\pm$ 0.6	0.0 $\pm$ 0.0
metadata reorder	Fixed XOR	3/3	11.7 $\pm$ 1.2	8.0 $\pm$ 0.0
metadata reorder	Adaptive	3/3	13.0 $\pm$ 1.0	14.3 $\pm$ 3.8

6 Discussion

6.1 Separate the probability model, estimator, and transport objective

The calibration experiment validates a decoder probability calculation under known erasure parameters. The policy experiment adds estimation from finite histories. The transport experiments add serialization, feedback delay, timers, and finite files. Success at one layer is insufficient evidence for the next. A controller can satisfy a model probability threshold while missing its true risk because the estimate or model is wrong. It can also reduce recovery work while spending more useful capacity on repairs.

Minimizing repair ratio subject to a block-risk constraint does not maximize file goodput. The objective omits the waiting time avoided by repair, feedback lag, packet scheduling, and the queueing cost of extra packets. The mixed baseline results are consistent with that objective mismatch, but this descriptive study does not isolate a unique causal explanation for every difference. A prospective goodput controller would need comparable tuning budgets, a declared optimization target, and evaluation on conditions excluded from its development.

6.2 Decoder structure and ordering matter at a fixed payload budget

Equal parity bytes do not imply equal recovery behavior. Row/column equations and replicas assign the same budget to different source relationships. Their response also depends on where correlated erasures fall in the source/repair sequence. The paired experiment makes that distinction visible without confounding it with additional redundancy. It supplies a concrete regression study for code/ordering changes, rather than a recommendation that one small code should replace another across networks.

The four-corner stopping example follows directly from the implemented peeling equations. The prior 2-D parity literature already analyzes deadlocks and residual packet loss [2]. Our contribution is the artifact’s explicit connection between that decoder, its block-risk calculations, fitted-policy diagnostics, and finite-transfer observations. The exact enumeration and factor envelope are transparent tools for this scope, not a claim to a new coding family or a broadly optimal controller.

6.3 Congestion and control traffic belong in the accounting

Coding does not remove a repair packet’s cost at a bottleneck. RFC 9265 describes this interaction and the risk of changing congestion signals when FEC hides source loss [6]. Our shared-FIFO experiment charges repairs to the same flight budget and reports useful release rather than requested rate. Its finite-file fairness results should not be interpreted as long-run equilibrium fairness.

Likewise, successful decoding assumes that the receiver knows the block’s membership and original length. The idealized emulator control path is useful for isolating transport behavior, but the socket protocol tests an impaired manifest and descriptors. Reporting these paths separately prevents reliable data delivery from being credited to an untested loss-free setup exchange. Reliable control increases the traffic and timing boundary; it is part of the protocol, not invisible preprocessing.

7 Validity and limitations

Internal validity. Known-probability risk enumeration and byte decoding share equation membership, so agreement does not independently validate the encoder specification itself. The correctness tests include hand-constructed erasure patterns, partial tails, and binary payloads, while calibration separately records every mask and observed outcome. Baseline schemes consume dif-

ferent loss draws after their schedules diverge. Transport differences can reflect that variation as well as scheme behavior. The small seed budgets support descriptive condition-specific statements. The online optional burst and uncertainty policies are not compared in the baseline goodput sweep; their decoder-level diagnostics cannot establish end-to-end gains.

Statistical validity. Individual Wilson intervals are approximate. Markov transition counts have random departure-state sample sizes, and nominal adjustment does not prove finite-sample simultaneous coverage. The policy diagnostics are multiple comparisons over reused parameter settings and training histories. We report their denominators without a calibrated family-wise interpretation. Equal-budget burst blocks are correlated; whole-seed paired contrasts avoid treating them as independent binomial trials, but five trace replications leave uncertain close rankings. Endpoints and zero observed failures do not prove zero risk outside the modeled settings.

Construct validity. Unrepaired-block risk, residual packet fraction, complete-file goodput, in-order release, and sender completion are different endpoints. Repair/file ratios use payload bytes and padding; emulator wire overhead uses fixed header assumptions. Socket counters measure actual framed UDP payload bytes but omit IP/UDP headers. Virtual-time and wall-clock measurements cannot be pooled. CRC and SHA-256 checks concern accidental integrity and reconstruction; neither supplies cryptographic peer authentication.

External validity. The loss models are synthetic and packet-indexed. Real loss can depend on queue occupancy, link layer retries, time, path changes, and hidden states with probabilistic emissions. The fitted envelope assumes stationary binary Markov loss with identical source/repair erasure rules and the specified order. Missing or delayed feedback and abrupt parameter changes break that premise. No cellular, WAN, multipath, or Internet field trial is performed. The socket transport binds to loopback and has no authenticated encryption, path-MTU discovery, or production congestion control. Its file sizes and the teaching AIMD queue constrain any deployment interpretation.

Research scope. This paper reports a reproducible empirical artifact and negative/mixed findings. It does not compare measured performance against AC-RLNC, FIEC, Reed-Solomon, or a deployed QUIC stack. Those methods have different objectives and implementations. The literature search is a bounded positioning exercise, with incomplete automated citation-graph coverage due to index rate limits, rather than evidence for an exhaustive novelty claim. A restricted contemporary bibliography also does not replace a historical account of ARQ and parity coding.

8 Conclusion

Decoder-specific residual risk, uncertainty in loss estimates, and useful transport throughput require separate evaluation. In the configured artifact, exact independent and binary Markov calculations agree with independently initialized byte-decoder trials, while fitted targets can fail when the estimate or process model is wrong. The saved equal-budget experiment changes the observed ordering of a small grid and replication between independent and burst erasures. Finite-transfer comparisons also preserve conditions where adaptive parity lowers mean retries while fixed parity has higher mean goodput. ParityLab makes these distinctions reproducible through masks, raw records, source fingerprints, and separate protocol execution checks. Extending the findings to a deployed transport requires external traces, stronger coding baselines, and a prospective congestion-aware evaluation.

Declarations

Author information. Dr. Ranjithkumar S provided mentorship and supervision. The remaining contribution details, corresponding-author contact, approval of the final manuscript by both

authors, and conflicts of interest await confirmation. This complete technical manuscript is not a venue-approved submission candidate.

Funding. This work received no funding.

Data and code availability. The source, saved experiments, project reports, and manuscript package are available at [the ParityLab repository](#), under the repository’s MIT license. The measured implementation revision is `1ee75227af7288491722a42813203a136ed37948`. The manuscript package records its own analysis inputs and outputs without changing that measured revision. No third-party article PDF is redistributed in the artifact.

Ethics and restrictions. The reported experiments use synthetic bytes and local simulated/loopback traffic; they contain no participants, personally identifying network traces, or animal work. The authors should confirm any institutional or target-venue ethics requirement. No claim of institutional approval or exemption is made. Original college register numbers and faculty submission fields are omitted from the public experiment documents; the research affiliation supplied for this paper is retained.

AI assistance. An AI assistant supported implementation, debugging, literature retrieval, analysis scripting, figure preparation, and manuscript drafting and editing. The experimental records and mathematical claims have machine checks documented in the artifact. Human authors remain responsible for reviewing the methods, citations, results, attribution, and final text; this disclosure does not claim that final author review has already occurred. A selected venue may require a different placement or more detailed declaration.

References

- [1] V. Roca and B. Teibi. Sliding Window Random Linear Code (RLC) Forward Erasure Correction (FEC) Schemes for FECFRAME. RFC 8681, January 2020. DOI: 10.17487/RFC8681. <https://www.rfc-editor.org/rfc/rfc8681.html>
- [2] F. Golaghaezadeh, S. Coulombe, and J.-M. Robert. Residual packet loss rate analysis of 2-D parity forward error correction. *Signal Processing: Image Communication* 102, 116597, March 2022. DOI: 10.1016/j.image.2021.116597. <https://doi.org/10.1016/j.image.2021.116597>
- [3] F. Michel, A. Cohen, D. Malak, Q. De Coninck, M. Medard, and O. Bonaventure. FIEC: Enhancing QUIC With Application-Tailored Reliability Mechanisms. *IEEE/ACM Transactions on Networking* 31(2), 606-619, April 2023. DOI: 10.1109/TNET.2022.3195611. Author manuscript: arXiv:2208.07741 (2022). <https://doi.org/10.1109/TNET.2022.3195611>
- [4] A. Cohen, D. Malak, V. Bar Bracha, and M. Medard. Adaptive Causal Network Coding With Feedback. *IEEE Transactions on Communications* 68(7), 4325-4341, July 2020. DOI: 10.1109/TCOMM.2020.2989827. <https://doi.org/10.1109/TCOMM.2020.2989827>
- [5] J. Iyengar and I. Swett. QUIC Loss Detection and Congestion Control. RFC 9002, May 2021. DOI: 10.17487/RFC9002. <https://www.rfc-editor.org/rfc/rfc9002.html>
- [6] N. Kuhn, E. Lochin, F. Michel, and M. Welzl. Forward Erasure Correction (FEC) Coding and Congestion Control in Transport. RFC 9265, July 2022. DOI: 10.17487/RFC9265. <https://www.rfc-editor.org/rfc/rfc9265.html>
- [7] J. Uberti. WebRTC Forward Error Correction Requirements. RFC 8854, January 2021. DOI: 10.17487/RFC8854. <https://www.rfc-editor.org/rfc/rfc8854.html>
- [8] G. Zeng, L. Chen, B. Yi, and K. Chen. Optimizing Tail Latency in Commodity Datacenters using Forward Error Correction. arXiv:2110.15157, October 2021. DOI: 10.48550/arXiv.2110.15157. <https://arxiv.org/abs/2110.15157>
- [9] D. Moskvitin, E. Onegin, R. Huang, H. Luo, and Q. Chen. Adaptive Forward Erasure Correction for Delay-Sensitive QUIC Connections. Internet-Draft draft-dmoskvitin-quic-adaptive-fec-00, May 6, 2024. This version expired November 7, 2024; work in progress, not an adopted standard. <https://www.ietf.org/archive/id/draft-dmoskvitin-quic-adaptive-fec-00.html>
- [10] T. Tsubaki, S. Anno, S. Komatsu, T. Torii, and T. Tojo. Scheduler-Agnostic Adaptive-FEC for MPQUIC: Field Evaluation over Commercial Cellular Paths. arXiv:2607.14482v1, July 16, 2026. DOI: 10.48550/arXiv.2607.14482. <https://arxiv.org/abs/2607.14482>

A Reproducibility and evidence map

A.1 Source, environment, and commands

Recorded measurements use Python 3.10.11. The core implementation and correctness suite use the standard library. Optional plot/report dependencies are pinned in `requirements-reproduction.txt`; the paper builder requires the plotting and PDF inspection environment. Installation supports Python 3.10 or newer. The source manifests hash the exact measured file bytes; line endings are preserved by the repository attributes. These hashes, rather than an unverified version label, establish which implementation the saved evidence describes.

```
python -m pip install -e .
python -m pip install -r requirements-reproduction.txt
python scripts/verify.py
python scripts/build_paper.py
```

The last command reconstructs tables from raw records, checks shared-mask pairing, derives seed contrasts, draws five figures, and writes a self-contained LaTeX source with embedded vector plots and bibliography. It does not rerun the original experiments. Open `paper/main.tex` in a standalone LaTeX editor to compile and review it. An existing local `pdflatex` can export the same file through `python scripts/build_paper.py --pdf`; no TeX installation is bundled or downloaded. SVG and PNG figures remain available for other venue templates.

To regenerate the underlying studies instead of reusing them:

```
python scripts/reproduce.py --calibration
python scripts/verify.py
python scripts/build_paper.py
```

Full calibration includes millions of decoder trials. Saved experiment records do not preserve reliable CPU time or peak memory for every original run, so no original compute-cost claim is made. New reproduction can log elapsed time on the operator’s machine; socket time and masks may differ with process scheduling. The deterministic payload generator uses seed 12345. Calibration seed derivation is $1000003 + \text{case_index} * 100003 + \text{seed}$. Policy training uses $8000003 + \text{seed} * 100003 + \text{history_length}$, and test blocks use $19000003 + \text{seed} * 100003 + \text{history_length}$. Equal-budget trials use the declared seed directly. All configured raw records are retained; study budgets are fixed rather than extended after inspecting a result.

A.2 Claim-to-artifact map

Claim in this manuscript	Primary artifact	Scope/check
Exact decoder calibration, Table 3 / Figures 2-3	output/calibration/summary.csv, raw-trials.csv.gz, seed-counts.csv	153 conditions; raw counts/outcomes and source digests audited
Estimated-policy failures, Table 4	output/calibration/policy-validation.csv	Histories, training masks, fitted parameters, chosen code and fresh test counts
Goodput/retry tradeoffs, Table 2 / Figure 1	output/experiments/runs.json, metrics.csv, summary.csv	260 transfers; complete output and condition-specific seed summaries
Equal-budget ordering, Table 5 / Figure 4	output/studies/equal-overhead/trials.csv.gz, summary.csv	Exact within-pair masks; independent traces and continuous burst traces separated
Shared capacity, Table 6 / Figure 5	output/congestion/runs.json.gz, summary.json	70 runs, 140 byte-verified flows; common-interval release and queue checks
Protocol execution, Table 7	output/socket-evaluation/runs.json, per-run receipts and output bytes	36 completed loopback transfers; three distinct processes per transfer
Parameter sensitivity, Appendix B	output/studies/sensitivity/runs.json, summary.csv	370 one-factor transfers; fixed-block sweep only affects fixed XOR

The paper’s `analysis.json`, `plot-data.json`, and `manifest.json` record the reconstruction and display inputs. The current audit reconstructs raw counts and checks measured source fingerprints. The older `output/calibration/verification.json` is a sampled replay receipt from before the latest full calibration regeneration; it is not represented as a replay of the current masks. The current raw-data audit and calibration manifest are the relevant provenance checks for this manuscript.

B Complete one-factor sensitivity summaries

Table B1. All sensitivity groups. Values are mean $\pm$ sample SD over five seeds. File size is 64 KiB. Factor names retain units: delay and timeout in milliseconds, capacity in Mbps, packet size in bytes, window and fixed block size in packets, and mean burst run in transmitted symbols. The baseline row uses independent loss 10%, delay 50 ms, capacity 5 Mbps, window 32, packet width 1,024 bytes and fixed block size eight. Each listed condition changes only its stated factor.

Factor	Value	Scheme	Goodput (Mbps)	Data retries
baseline	baseline	GBN	0.360 ± 0.120	146.4 ± 63.7
baseline	baseline	SR	1.003 ± 0.342	6.2 ± 2.6
baseline	baseline	Fixed XOR	1.361 ± 0.593	3.6 ± 2.6
baseline	baseline	Adaptive	1.104 ± 0.284	5.0 ± 1.6
delay ms	0	GBN	0.748 ± 0.251	146.4 ± 63.7
delay ms	0	SR	2.401 ± 0.794	6.2 ± 2.6
delay ms	0	Fixed XOR	3.001 ± 0.940	3.6 ± 2.6
delay ms	0	Adaptive	2.622 ± 0.574	5.2 ± 1.5
delay ms	25	GBN	0.496 ± 0.173	146.4 ± 63.7
delay ms	25	SR	1.457 ± 0.525	6.2 ± 2.6
delay ms	25	Fixed XOR	2.001 ± 0.921	3.6 ± 2.6
delay ms	25	Adaptive	1.595 ± 0.423	5.0 ± 1.6
delay ms	100	GBN	0.232 ± 0.074	146.4 ± 63.7
delay ms	100	SR	0.617 ± 0.199	6.2 ± 2.6
delay ms	100	Fixed XOR	0.831 ± 0.345	3.6 ± 2.6
delay ms	100	Adaptive	0.683 ± 0.171	5.0 ± 1.6
bandwidth mbps	1	GBN	0.128 ± 0.044	146.4 ± 63.7
bandwidth mbps	1	SR	0.372 ± 0.125	6.2 ± 2.6
bandwidth mbps	1	Fixed XOR	0.495 ± 0.194	3.6 ± 2.6
bandwidth mbps	1	Adaptive	0.420 ± 0.096	5.2 ± 1.5
bandwidth mbps	2	GBN	0.217 ± 0.075	146.4 ± 63.7
bandwidth mbps	2	SR	0.616 ± 0.211	6.2 ± 2.6
bandwidth mbps	2	Fixed XOR	0.874 ± 0.383	3.6 ± 2.6
bandwidth mbps	2	Adaptive	0.714 ± 0.171	4.8 ± 1.8
bandwidth mbps	10	GBN	0.458 ± 0.146	146.4 ± 63.7
bandwidth mbps	10	SR	1.222 ± 0.396	6.2 ± 2.6
bandwidth mbps	10	Fixed XOR	1.652 ± 0.694	3.6 ± 2.6
bandwidth mbps	10	Adaptive	1.353 ± 0.340	5.0 ± 1.6
window	8	GBN	0.365 ± 0.073	46.4 ± 20.4
window	8	SR	0.431 ± 0.062	6.2 ± 2.6
window	8	Fixed XOR	0.460 ± 0.092	3.4 ± 2.3
window	8	Adaptive	0.498 ± 0.034	2.2 ± 0.4
window	16	GBN	0.348 ± 0.082	120.6 ± 38.5
window	16	SR	0.599 ± 0.133	6.2 ± 2.6
window	16	Fixed XOR	0.755 ± 0.287	3.4 ± 2.3
window	16	Adaptive	0.651 ± 0.066	4.2 ± 1.1
window	64	GBN	0.256 ± 0.048	252.4 ± 95.6

Table continued.

Factor	Value	Scheme	Goodput (Mbps)	Data retries
window	64	SR	1.110 ± 0.320	6.2 ± 2.6
window	64	Fixed XOR	1.726 ± 0.763	3.4 ± 2.4
window	64	Adaptive	1.110 ± 0.320	6.2 ± 2.6
timeout ms	70	GBN	0.887 ± 0.265	188.8 ± 61.7
timeout ms	70	SR	1.413 ± 0.320	71.2 ± 3.5
timeout ms	70	Fixed XOR	1.730 ± 0.299	69.0 ± 4.5
timeout ms	70	Adaptive	1.596 ± 0.274	69.4 ± 2.7
timeout ms	180	GBN	0.371 ± 0.123	146.4 ± 63.7
timeout ms	180	SR	1.025 ± 0.345	6.2 ± 2.6
timeout ms	180	Fixed XOR	1.379 ± 0.585	3.6 ± 2.6
timeout ms	180	Adaptive	1.126 ± 0.285	5.0 ± 1.6
timeout ms	500	GBN	0.149 ± 0.056	146.4 ± 63.7
timeout ms	500	SR	0.511 ± 0.226	6.2 ± 2.6
timeout ms	500	Fixed XOR	0.937 ± 0.795	3.6 ± 2.6
timeout ms	500	Adaptive	0.588 ± 0.200	5.0 ± 1.6
packet size	512	GBN	0.250 ± 0.034	356.0 ± 80.9
packet size	512	SR	0.524 ± 0.114	15.6 ± 3.6
packet size	512	Fixed XOR	0.737 ± 0.146	9.6 ± 3.9
packet size	512	Adaptive	0.691 ± 0.062	7.6 ± 2.2
packet size	1200	GBN	0.564 ± 0.297	109.2 ± 74.6
packet size	1200	SR	1.060 ± 0.238	5.2 ± 1.8
packet size	1200	Fixed XOR	1.516 ± 0.514	2.8 ± 1.8
packet size	1200	Adaptive	1.117 ± 0.216	4.8 ± 1.5
fixed k	4	Fixed XOR	1.455 ± 0.539	2.0 ± 1.9
fixed k	16	Fixed XOR	1.154 ± 0.255	5.0 ± 2.2
burst mean packets	2	GBN	0.928 ± 0.405	59.0 ± 50.4
burst mean packets	2	SR	1.295 ± 0.229	4.2 ± 4.0
burst mean packets	2	Fixed XOR	1.619 ± 0.677	2.6 ± 3.2
burst mean packets	2	Adaptive	1.239 ± 0.336	4.6 ± 4.8
burst mean packets	5	GBN	1.684 ± 0.827	21.2 ± 20.1
burst mean packets	5	SR	1.846 ± 0.639	4.8 ± 4.8
burst mean packets	5	Fixed XOR	1.703 ± 0.589	4.0 ± 4.0
burst mean packets	5	Adaptive	1.816 ± 0.670	4.8 ± 4.8
burst mean packets	10	GBN	1.927 ± 0.873	13.8 ± 20.1
burst mean packets	10	SR	2.072 ± 0.644	3.6 ± 5.4
burst mean packets	10	Fixed XOR	1.942 ± 0.557	3.2 ± 4.9
burst mean packets	10	Adaptive	2.066 ± 0.653	3.6 ± 5.4

C The 50% parity-budget comparison

Table C1. Residual failures for two equal-width repairs over four sources. Each entry is failed blocks / 5,000 shared masks across five seeds. Both codes within a pair use the same source-then-repair ordering. Continuous Markov traces use a five-symbol mean bad run; their blocks are correlated. Counts supplement the 100% budget comparison rather than implying a budget-independent ranking.

Model	Loss	Pairwise XOR	Repeated block XOR
IID	1%	1/5000	3/5000
IID	5%	74/5000	93/5000
IID	10%	273/5000	298/5000
IID	20%	962/5000	941/5000
Markov	1%	53/5000	53/5000
Markov	5%	324/5000	333/5000
Markov	10%	573/5000	595/5000
Markov	20%	1188/5000	1223/5000